

Awesome—let's get you from **beginner to career-ready in AI/ML**. Since you didn't fill the context fields, I'll assume a **beginner starting point**, a **balanced mix of theory + practice**, and give **time-boxed variants** so you can match your schedule:

- **Fast-track:** ~6–8 months (full-time)
- **Standard:** ~9–12 months (15–20 hrs/week)
- **Part-time:** ~15–18+ months (8–12 hrs/week)

You can plug in your exact context later; the roadmap will still work.

Full Roadmap (Beginner → Career-Ready)

Phase 1: Programming & Math Foundations (Weeks 1–10)

Objectives

- Python fluency for data work; command line, Git/GitHub
- Math for ML: functions, calculus basics, linear algebra, probability/statistics intuition

Core Curriculum

- **Python:** *Python for Everybody* (Coursera) or CS50P; *Automate the Boring Stuff* (book)
- **Math:** 3Blue1Brown (Linear Algebra, Calculus), Khan Academy Probability & Stats; *The Matrix Calculus You Need For Deep Learning* (paper/notes)
- **Tools:** Jupyter/VS Code, Git/GitHub, NumPy, Pandas, Matplotlib

Hands-On Projects (pick 2–3)

1. **Data Story:** Clean a messy CSV (Kaggle public dataset), answer 5 questions, plot 6 figures.
2. **Automation:** A CLI that tidies your files or scrapes + summarizes news to CSV.

3. **Mini-Stats Lab:** Simulate coin flips/dice; estimate probabilities & confidence intervals.

Success Metrics

- Write/read clean Python modules, use virtualenv/poetry, push to GitHub
 - Explain mean/variance, conditional probability, matrix multiply, gradients idea
 - Reproduce data cleaning + EDA on a novel dataset in under 2 hours
-

Phase 2: Core Machine Learning (Weeks 11–18)

Objectives

- Supervised/unsupervised learning, model evaluation, bias/variance
- Feature engineering, cross-validation, pipelines

Core Curriculum

- **Courses:** Andrew Ng's *Machine Learning Specialization* (Coursera) **or** *Introduction to Machine Learning* (Udacity)
- **Book:** *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow* (Géron) — focus on Parts 1–2 (Scikit-learn)
- **Tools:** scikit-learn, XGBoost/LightGBM, MLflow for experiments

Projects

1. **Tabular ML Baselines:** Compare 5 models on a tabular dataset; build a scikit-learn Pipeline with cross-val; log runs in MLflow.
2. **Text or Time-Series Classifier:** TF-IDF + Logistic Regression for sentiment **or** ARIMA/Prophet baseline for forecasting.
3. **Kaggle “Getting Started”:** Submit ≥ 2 competitions (Titanic/House Prices), create a public notebook with narrative EDA.

Success Metrics

- Explain train/val/test, ROC-AUC vs accuracy, regularization, leakage
 - Build a full ML pipeline with CV and hyperparameter search
 - Achieve top 25–30% on a beginner Kaggle comp without copying
-

Phase 3: Deep Learning Foundations (Weeks 19–28)

Objectives

- Backprop, optimization (SGD, Adam), initialization, normalization
- Build neural networks from scratch; use a modern DL framework

Core Curriculum

- **Courses:** fast.ai *Practical Deep Learning for Coders* (v2+), DeepLearning.AI *Deep Learning Specialization*
- **Books:** *Dive Into Deep Learning (D2L)* (interactive) + Chapters 6–8 of Géron (Keras/TensorFlow)
- **Frameworks:** PyTorch (primary) or TensorFlow/Keras (secondary)
- **Tools:** Weights & Biases or MLflow, CUDA basics (if GPU access)

Projects

1. **MNIST From Scratch:** NumPy-only MLP + backprop; then PyTorch version, compare learning curves.
2. **Image Classifier (Transfer Learning):** Fine-tune ResNet on a custom dataset (e.g., plant disease, fabric defects).
3. **NLP Classifier:** Fine-tune a small transformer (DistilBERT) for sentiment or topic classification.

Success Metrics

- Implement forward/backward for a 2–3 layer MLP; plot loss/metrics
 - Train/fine-tune a CNN with data augmentation; achieve $\geq 90\%$ on CIFAR-10 (transfer)
 - Clearly explain vanishing gradients, overfitting/regularization, and learning rate schedules
-

Phase 4: MLOps & Productization (Weeks 29–36)

Objectives

- Reproducibility, packaging, testing, data/versioning, serving/monitoring
- Lightweight deployment patterns

Core Curriculum

- **Concepts:** Docker, unit tests/pytest, pre-commit/ruff/black, DVC, MLflow, feature stores (high level), batch vs. real-time serving
- **Micro-courses:** Full Stack Deep Learning (MLOps weeks), Made With ML (MLOps)

Projects

1. **Model Service:** Train model → package as Docker → REST API (FastAPI) → simple frontend (Streamlit/Gradio).
2. **Repro Pipeline:** DVC for data + MLflow for experiments; one-click retrain script.
3. **Monitoring Demo:** Log inference latency + drift metrics; set alert thresholds.

Success Metrics

- `docker build/run` works on a teammate's machine; README enables 15-min setup
- Canary or blue/green simulation locally; rollback script works

- Basic data/feature drift dashboard that updates from logs
-

Phase 5: Specialization (Weeks 37–48+)

Choose **one or two** based on interest (see tracks below). Expect 10–12 weeks per track.

Common Advanced Readings

- *Deep Learning* (Goodfellow) — Chapters on CNNs/RNNs/optimization
- Key papers relevant to your track (listed below)

Capstone

- End-to-end, **portfolio-ready** project solving a real problem with deployment + monitoring + write-up.

Success Metrics

- One polished capstone with live demo (or recorded video), clear README, tests, and a blog post
 - Reference-check ready: peers can run, evaluate, and extend your work
-

Specialization Tracks (pick what fits your interests)

A) Generative AI & LLMs

Focus

- Tokenizers, attention, pretraining vs. fine-tuning (LoRA/QLoRA), retrieval-augmented generation (RAG), evaluation (hallucinations, toxicity)

Core

- **Courses:** DeepLearning.AI *LLM Engineering*, *RAG*, *Prompt Engineering* tracks; fast.ai NLP lessons
- **Tools:** Hugging Face Transformers/Datasets, PEFT, bitsandbytes, Faiss/Chroma, LangChain/LlamaIndex, Guardrails
- **Key Papers:** *Attention Is All You Need* (2017), *BERT* (2018), *LoRA* (2021), *RAG* frameworks (2020+)

Projects

1. **Domain RAG Assistant:** Index PDFs/website; evaluate with question-answer pairs; add guardrails + feedback logging.
 2. **Parameter-Efficient Fine-Tune:** QLoRA fine-tune a small LLM on a domain dataset; compare to zero-shot.
 3. **Generation Safety & Eval:** Build automatic evals (faithfulness, toxicity, latency, cost) and a dashboard.
-

B) Computer Vision

Focus

- CNNs, modern vision transformers (ViT), detection/segmentation, augmentation, labeling tools

Core

- **Tools:** PyTorch/TensorFlow, Albumentations, YOLOv8/Detectron2, Lightning
- **Papers:** *ResNet* (2015), *U-Net* (2015), *Mask R-CNN* (2017), *ViT* (2020)

Projects

1. **Custom Detector:** YOLOv8 for a real-world detection task; label with Roboflow/Label Studio; export to ONNX.
 2. **Segmentation for Quality Control:** U-Net for defects; add uncertainty estimation (MC dropout) to flag low-confidence cases.
 3. **Mobile Inference:** Quantize & run on CPU/mobile; measure accuracy-latency trade-offs.
-

C) Reinforcement Learning (applied)

Focus

- MDPs, value vs. policy methods, function approximation, simulation tooling

Core

- **Courses:** Spinning Up in Deep RL (OpenAI), Hugging Face Deep RL Course
- **Tools:** Gymnasium, Stable-Baselines3, RLlib
- **Papers:** *DQN* (2015), *PPO* (2017), *SAC* (2018)

Projects

1. **Classic Control Benchmarks:** Solve CartPole/LunarLander; analyze sample efficiency vs PPO/SAC.
 2. **Sim2Real-ish:** Train a warehouse-picking or scheduling policy in sim; add domain randomization.
 3. **Safety/Reward Shaping:** Compare reward shaping vs. constraint methods; produce learning-curve diagnostics.
-

D) AI for Finance (or Healthcare)

Focus

- Time-series forecasting, anomaly detection, risk modeling (Finance) **or** clinical NLP/CV with privacy (Healthcare)

Core

- **Finance Tools:** statsmodels, gluonts/pytorch-forecasting, feature stores; backtesting protocols (walk-forward)
 - **Healthcare Tools:** de-identification, simple clinical NLP (entity extraction), weak supervision (Snorkel)
 - **Projects**
 - **Finance:** Multi-horizon forecasting with backtest; trading rules + slippage modeling; robust evaluation.
 - **Healthcare:** Clinical text classifier (de-identified), RAG over guidelines with safety filters; calibration plots.
-

Portfolio Building (do this throughout)

- **GitHub hygiene:** One repo per project, clean **README.md** (goal, data, how to run, results), tags/releases, unit tests, **requirements.txt** or **pyproject.toml**, Dockerfile.
 - **Project polish:** Record a **2–3 min demo video**; include screenshots, metrics tables, and ablations.
 - **Blog/Notes:** Publish learnings on Hashnode/Medium/GitHub Pages. Aim for 1 post per phase (e.g., “Bias-variance hands-on,” “From notebook to Docker”).
 - **Kaggle:** 2–3 public notebooks with narrative + reproducibility; 1 competition with top-25% finish or a gold-standard EDA.
 - **Data ethics:** Add a short *Model Card / Data Card* to each repo.
-

Career Prep

What to learn for interviews

- **Coding:** Arrays/strings, hash maps, two-pointers, BFS/DFS, heaps, sorting, binary search, recursion (LeetCode-style). Target: 60–100 curated problems.
- **ML Theory:** Linear/logistic regression, trees/ensembles, bias/variance, regularization, loss functions, optimization, CNNs/RNNs/Transformers basics.
- **MLOps/System Design:** Data ingestion, feature stores, training/serving, A/B testing, monitoring, drift handling, retraining triggers, cost awareness.

Practice plan (8–12 weeks)

- 3 days/week coding (1–2 problems/day)
- 2 days/week ML theory Q&A (create flashcards from your notes)
- 1 day/week system design (draw a serving diagram + failure modes)
- 1 mock interview/week with a peer; rotate coding/ML/system design.

Applications & Networking

- Tailor resume bullets to **impact + metrics** (“reduced inference p95 latency by 40% via quantization”).
- Build a **T-shaped profile**: broad ML + deep capstone in your specialization.
- Engage: contribute a small PR to an OSS repo (docs/tests count), post weekly learning snippets on LinkedIn/X, join 1–2 local/online study groups.

Continuous Learning Strategy

- **Cadence:** 70% doing / 20% reading / 10% sharing.

- **Stay current:** Follow Papers with Code leaderboards; subscribe to arXiv weekly digests for your subfield; watch top conference tutorials (NeurIPS/ICLR/CVPR).
 - **Refreshers:** Re-implement a classic model from scratch every 2–3 months (e.g., logistic regression, ResNet block, attention).
 - **Evaluate yourself quarterly:** New dataset, new task, hard deadline → ship a mini-project end-to-end.
-

Suggested Weekly Rhythm (standard pace, 15–20 hrs)

- **Mon–Thu (1.5–2h/day):** Course + notes + 1–2 exercises
 - **Fri (2h):** Project increments (EDA/modeling/unit tests)
 - **Sat (4h):** Deep work sprint (build/clean/ship)
 - **Sun (2h):** Write a short log/blog; plan next week
-

Milestones & Checkpoints

- **Week 4:** Python project + GitHub repo with tests
 - **Week 10:** EDA portfolio piece + math summary notes
 - **Week 18:** 2 tabular ML projects + 1 Kaggle submission
 - **Week 28:** CNN + NLP fine-tune demos with W&B reports
 - **Week 36:** Deployed model service (Docker + FastAPI)
 - **Week 48:** Capstone shipped + blog + demo video
-

How to Customize This to Your Context

Reply with your:

- **Starting Point** (e.g., complete beginner / some coding)
- **End Goal** (ML Engineer / Data Scientist / Research-oriented / Domain-applied)
- **Time Commitment** (hrs/week or a target date)
- **Learning Preference** (project-driven / theory-heavy / balanced)
- **Areas of Interest** (e.g., LLMs, CV, RL, healthcare, finance)

I'll instantly **tailor durations, course picks, and project ideas**, and suggest a **capstone concept** aligned to your goals.